

Lecture 8

Model Selection

Model Selection is the notion of choosing the **best** statistical model from a set of candidate models, given the data.

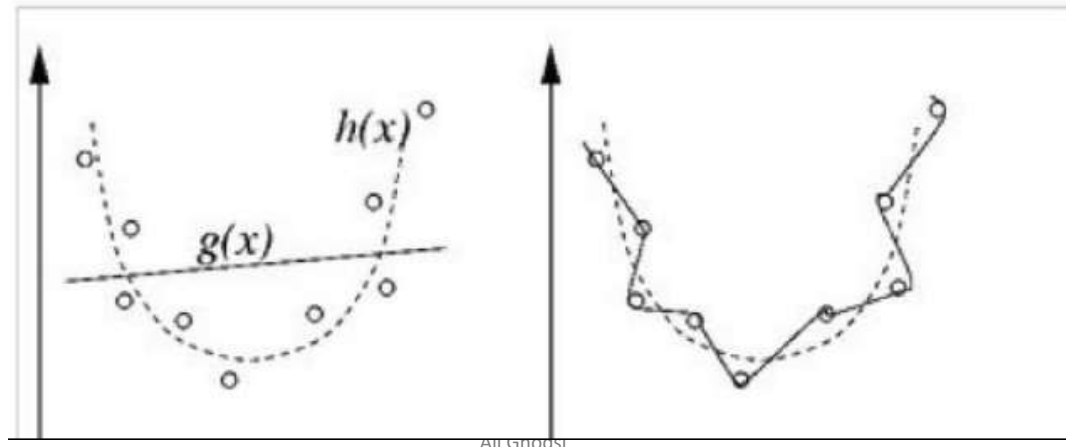
We would like to select a model (classifier) h which has a low true error rate $L(h) = Pr\{h(x) \neq y\}$.

However, we should avoid simply using the empirical error rate $\hat{L}(h) = \frac{1}{n} \sum_{i=1}^n I\{h(x_i) \neq y_i\}$ because it is biased downward.

Model Selection

Using the empirical error rate can find a model which simply returns $\hat{L}(h) = 0$ on the training set (right-hand figure). However, this is clearly not a good choice because the model overfits the training data and will not generalize to out-of-sample data.

On the other hand, it is also not good to choose a model which "over generalizes" (left-hand figure) because it tends to be far off from the true underlying process and will make poor predictions.



Model Selection

Left-hand model Advantage: Has much lower variance; given new data, the model parameterization will likely seldom change. Disadvantage: Has much higher bias; model estimates are far from the truth on average. These models are *low complexity* models; few parameters.

Right-hand model Advantage: Has much lower bias; Model estimates accurately predict training data. Disadvantage: Has much higher variance; given new data, the model parameterization will likely change drastically. These models are *high complexity* models; many parameters.

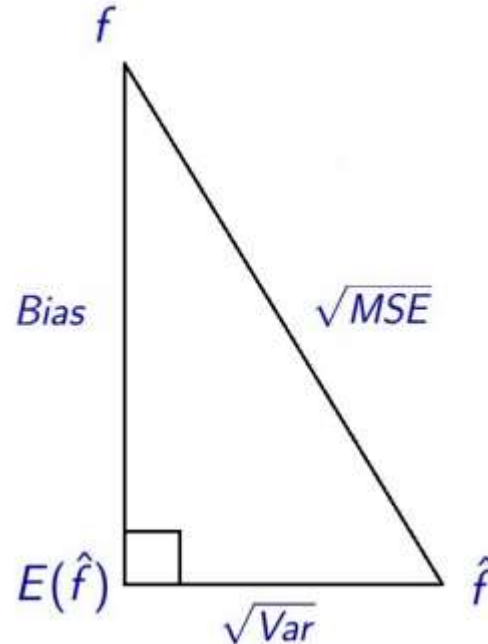
We refer to the **best** model as the one with optimal Bias-Variance tradeoff; the model with lowest Mean Squared Error (MSE).

Model Selection

$$Bias(\hat{f}) = E(\hat{f}) - f$$

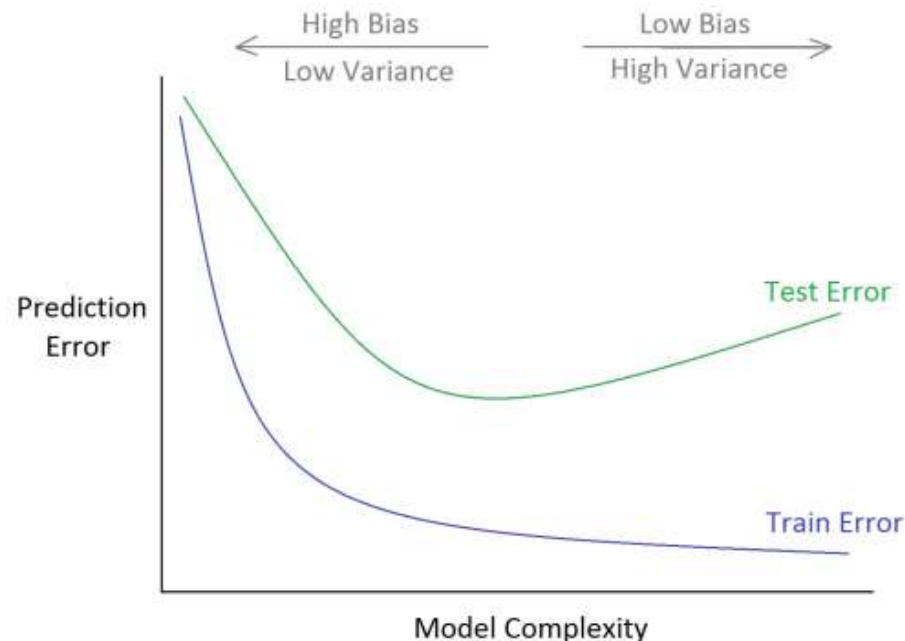
$$Var(\hat{f}) = E \left[(\hat{f} - E(\hat{f}))^2 \right]$$

$$MSE(\hat{f}) = E \left[(\hat{f} - f)^2 \right] = Bias^2(\hat{f}) + Var(\hat{f})$$



Model Selection

As we add more parameters to the model, we risk overfitting and thus the training error always decreases as we make the model more complex (good). On the other hand, a more complex model will start to predict poorly on out-of-sample data and thus the testing error will eventually start to increase after a certain point (bad). A reasonable model may be the one whose complexity minimizes the testing error.



Model Selection

There are two main strategies to avoid overfitting:

1) Estimate Error rate

- Cross Validation
- Compute error bound

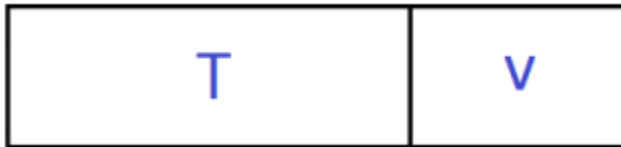
2) Regularization: Smoothing the function (model) by limiting the complexity or the size of weights

Cross Validation

Cross Validation

This is the practice of splitting the training set into two components, treating the first subset as actual training data and its complement as testing data.

The simplest type of cross validation involves randomly splitting the data into a training set $\mathcal{T}$, a validation set ν , and keeping them fixed.



The classifier h is estimated from the training set $\mathcal{T}$, and the true error is estimated to be:

$$\hat{L}(h) = \frac{1}{|\nu|} \sum_{x_i \in \nu} I\{h(x_i) \neq y_i\}$$

K-fold Cross Validation

Another type of cross validation is K-fold cross validation.

1) Randomly split the data into K chunks of roughly equal size (a standard choice is K=10).

2) For $k=1:K$ do

i) Remove chunk k from the data

ii) Compute classifier $\hat{h}_k$ from the rest of the data

iii) Use $\hat{h}_k$ to predict the data in chunk k and let $\hat{L}_k$ denote the observed error rate

3) Let $\hat{L}(h) = \frac{1}{K} \sum_{k=1}^K \hat{L}_k(\hat{h}_k)$

K-fold Cross Validation

The case when $K=4$:

T	T	T	V
T	T	V	T
T	V	T	T
V	T	T	T

Leave-One-Out Cross Validation

The third approach to cross validation is Leave-One-Out Cross validation (LOOCV). Here, we train the model a total of n times ($n = |Sample|$).

For $k=1:n$

- i) Temporarily remove the k^{th} data point
- ii) Estimate the classifier $\hat{h}_k$ on the remaining $n - 1$ points
- iii) Compute the prediction error on the k^{th} data point

After completing this for all n points, compute the mean error over all n points. This is the LOOCV estimate of the true error rate.

Regularization

A central problem in machine learning is how to make an algorithm that will perform well not just on the training data, but also on new inputs.

Most machine learning tasks are estimation of a function $\hat{f}(\mathbf{x})$ parametrized by a vector of parameters θ .

Regularization

A central problem in machine learning is how to make an algorithm that will perform well not just on the training data, but also on new inputs.

Most machine learning tasks are estimation of a function $\hat{f}(x)$ parameterized by a vector of parameters $\mathbf{w}$.

Classical Regularization: Parameter Norm Penalty

Most classical regularization approaches are based on limiting the capacity of models, by adding a parameter norm penalty $\Omega(\mathbf{w})$ to the objective function J .

$$J(\mathbf{w}; X, y) = J(\mathbf{w}; X, y) + \alpha\Omega(\mathbf{w})$$

L^2 Parameter Regularization, weight decay

L^2 parameter norm penalty commonly known as weight decay.

$$\text{Regularization term } \Omega(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|_2^2$$

Gradient of the total objective function:

$$\nabla_{\mathbf{w}} \tilde{J}(\mathbf{w}; X, y) = \alpha \mathbf{w} + \nabla_{\mathbf{w}} J(\mathbf{w}; X, y)$$

$$\mathbf{w} := \mathbf{w} - \epsilon(\alpha \mathbf{w} + \nabla_{\mathbf{w}} J(\mathbf{w}; X, y))$$

Considering a quadratic approximation to the objective function:

$$\hat{J}(\mathbf{w}) = J(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^T H(\mathbf{w} - \mathbf{w}^*)$$

$$\nabla_{\mathbf{w}} \hat{J}(\mathbf{w}) = H(\mathbf{w} - \mathbf{w}^*)$$

$$\alpha w + H(w - w^*) = 0$$

$$(H + \alpha I)w = Hw^*$$

$$\tilde{w} = (H + \alpha I)^{-1} Hw^*.$$

what happens as α grows?

$$H = Q\Lambda Q^T$$

$$\tilde{w} = (Q\Lambda Q^T + \alpha I)^{-1} Q\Lambda Q^T w^*$$

$$= [Q(\Lambda + \alpha I)Q^T]^{-1} Q\Lambda Q^T w^*$$

$$= Q(\Lambda + \alpha I)^{-1} \Lambda Q^T w^*,$$

$$Q^T \tilde{w} = (\Lambda + \alpha I)^{-1} \Lambda Q^T w^*.$$

The effect of weight decay is to rescale the coefficients of eigenvectors. The i th component is rescaled by a factor of $\frac{\lambda_i}{\lambda_i + \alpha}$.

- ▶ If $\lambda_i \gg \alpha$, the effect of regularization is relatively small.
- ▶ Components with $\lambda_i \ll \alpha$ will be shrunk to have nearly zero magnitude.

Directions along which the parameters contribute significantly to reducing the objective function are preserved a small eigenvalue of the Hessian tell us that movement in this direction will not significantly increase the gradient **effective number of parameters**, defined to be

$$\gamma = \sum_i \frac{\lambda_i}{\lambda_i + \alpha}.$$

As α is increased, the effective number of parameters decreases.

Dataset Augmentation

The best way to make a machine learning model generalize better is to train it on more data.

The amount of data we have is limited.

Create fake data and add it to the training set.

Not applicable to all tasks.

For example, it is difficult to generate new fake data for a density estimation task unless we have already solved the density estimation problem.

Dataset Augmentation

Operations like translating the training images a few pixels in each direction can often greatly improve generalization.

One way to improve the robustness of neural networks is simply to train them with random noise applied to their inputs.

This same approach also works when the noise is applied to the hidden units, which can be seen as doing dataset augmentation at multiple levels of abstraction.

Noise injection

Two ways that noise can be used as part of a regularization strategy.

Adding noise to the input.

This can be interpreted simply as form of dataset augmentation.

Can also interpret it as being equivalent to more traditional forms of regularization.

Noise injection

Two ways that noise can be used as part of a regularization strategy.

Adding noise to the input.

This can be interpreted simply as form of dataset augmentation.

Can also interpret it as being equivalent to more traditional forms of regularization.

Adding it to the weights.

This technique has been used primarily in the context of recurrent neural networks (Jim et al., 1996; Graves, 2011a).

This can be interpreted as a stochastic implementation of a Bayesian inference over the weights.

Early Stopping as a Form of Regularization

Instead of running our optimization algorithm until we reach a (local) minimum of validation error:

Run it until the error on the validation set has not improved for some amount of time.

Every time the error on the validation set improves, we store a copy of the model parameters.

When the training algorithm terminates, we return these parameters, rather than the latest parameters.

It is probably the most commonly used form of regularization in deep learning.

What is dropout?

- Dropout is one of the techniques for preventing overfitting in deep neural network which contains a large number of parameters.

Original Paper

- Title:
 - Dropout: A Simple Way to Prevent Neural Networks from Overfitting (2014).
- Authors:
 - Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov
- Organization:
 - Department of Computer Science, University of Toronto
- URL:
 - <https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf>

Overview

- The key idea is to randomly drop units from the neural network during training.
- During training, dropout samples from an exponential number of different “thinned” network.
- At test time, we approximate the effect of averaging the predictions of all these thinned networks.

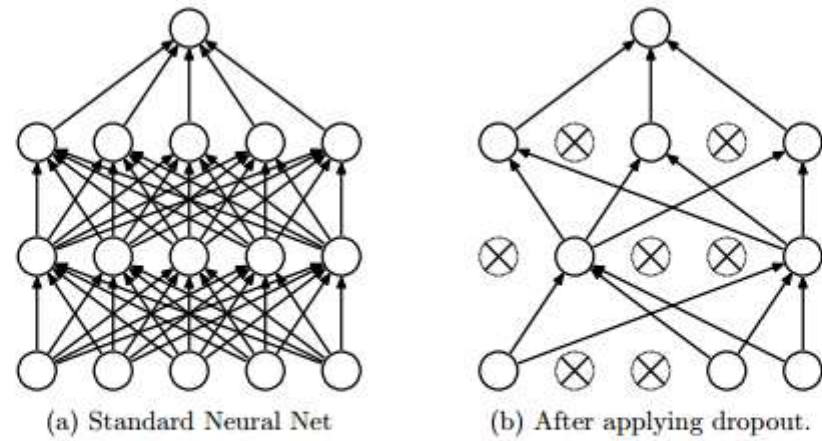


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Model

Consider a neural network with L hidden layer. Let $\mathbf{z}^{(l)}$ denote the vector inputs into layer l , $\mathbf{y}^{(l)}$ denote the vector of outputs from layer l . $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weights and biases at layer l . With dropout, the feed-forward operation becomes:

$$r_j^{(l)} \sim \text{Bernoulli}(p)$$

$$\tilde{\mathbf{y}}^{(l)} = \mathbf{r}^{(l)} * \mathbf{y}^{(l)}, \text{ here } * \text{ denotes an element-wise product.}$$

$$z_i^{(l+1)} = \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^l + b_i^{(l+1)}$$

$$y_i^{(l+1)} = f(z_i^{(l+1)}), \text{ where } f \text{ is the activation function.}$$

For any layer l , $\mathbf{r}^{(l)}$ is a vector of independent Bernoulli random variables each of which has probability of p of being 1. $\tilde{\mathbf{y}}$ is the input after we drop some hidden units. The rest of the model remains the same as the regular feed-forward neural network.

Training

- ▶ Dropout neural network can be trained using **stochastic gradient descent**.
- ▶ The only difference here is that we only back propagate on each thinned network.
- ▶ The gradient for each parameter are averaged over the training cases in each mini-batch.

Test Time

- ▶ use a single neural net without dropout.
- ▶ If a unit is retained with probability p during training, the outgoing weights of that unit are multiplied by p at test time.

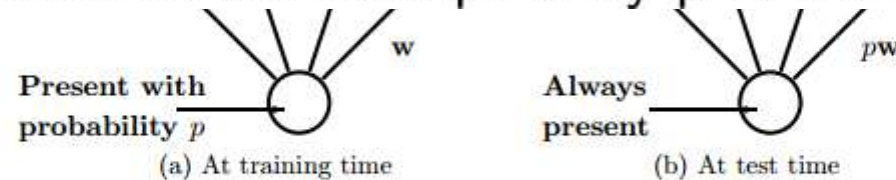


Figure 2: **Left:** A unit at training time that is present with probability p and is connected to units in the next layer with weights w . **Right:** At test time, the unit is always present and the weights are multiplied by p . The output at test time is same as the expected output at training time.

Keras: The Python Deep Learning library

Some slides courtesy of Aref Jafari

Step 1) Import Libraries

```
import numpy as np
import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout, Activation, Flatten, Input
from keras.utils import np_utils

#Other types of layers
from keras.layers import LSTM
from keras.layers import Conv1D, Conv2D, Conv3D, MaxPooling2D

from keras.layers.normalization import BatchNormalization

import matplotlib.pyplot as plt
%matplotlib inline

np.random.seed(2017)
```

Step 3) Define model architecture

Form 1)

```
In [11]: model = Sequential()  
model.add(Dense(512, activation='relu', use_bias=True, input_shape=(784,)))  
model.add(Dense(128, activation='relu', use_bias=True))  
model.add(Dense(10, activation='softmax', use_bias=True))
```

Form 2)

```
In [91]: from keras.models import Model  
  
X_inp = Input(shape=(784,))  
h1 = Dense(512, activation='relu', use_bias=True)(X_inp)  
h2 = Dense(128, activation='relu', use_bias=True)(h1)  
h3 = Dense(10, activation='softmax', use_bias=True)(h2)  
  
model = Model(inputs=X_inp, outputs=h3)
```

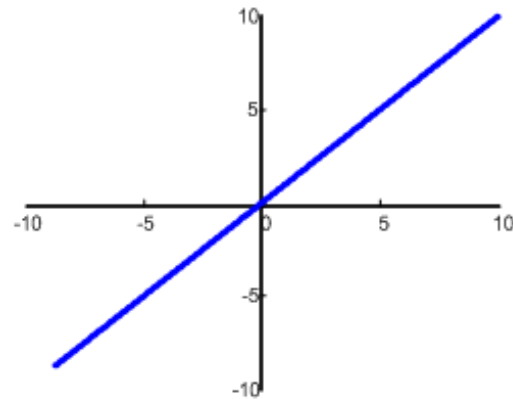
Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

linear

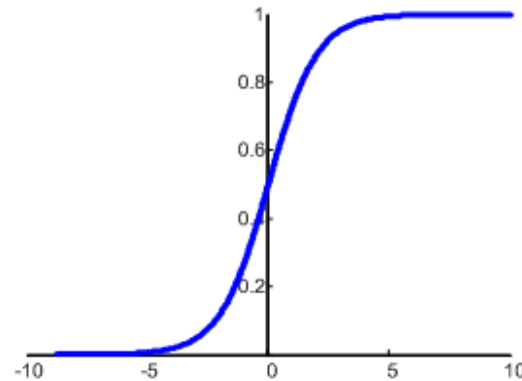


$$f(x) = x$$

Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

sigmoid

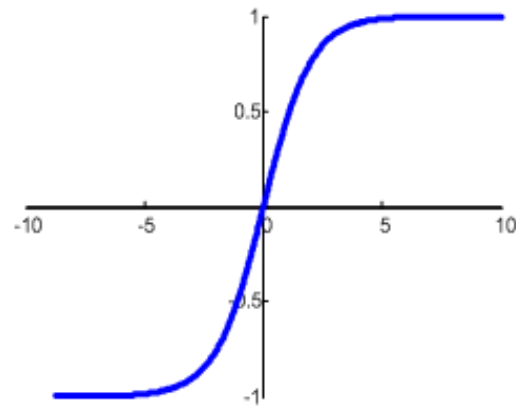


$$f(x) = \frac{1}{1 + e^{-x}}$$

Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

tanh

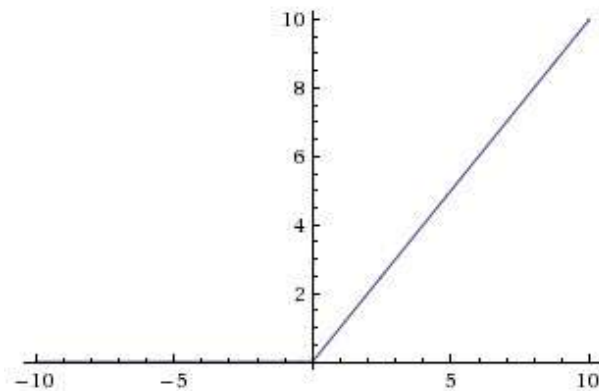


$$f(x) = \tanh(x)$$

Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

relu

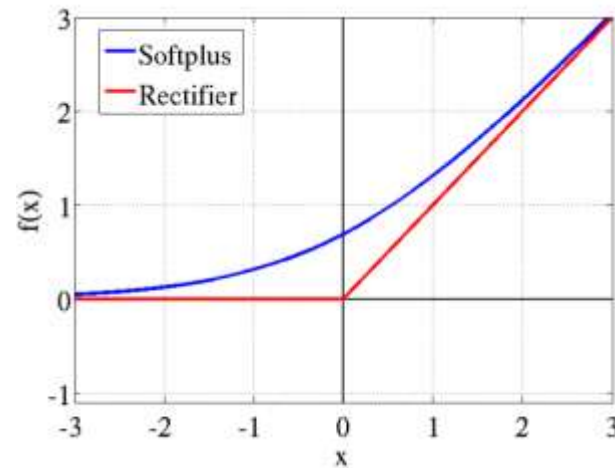


$$f(x) = \max(0, x)$$

Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

softplus

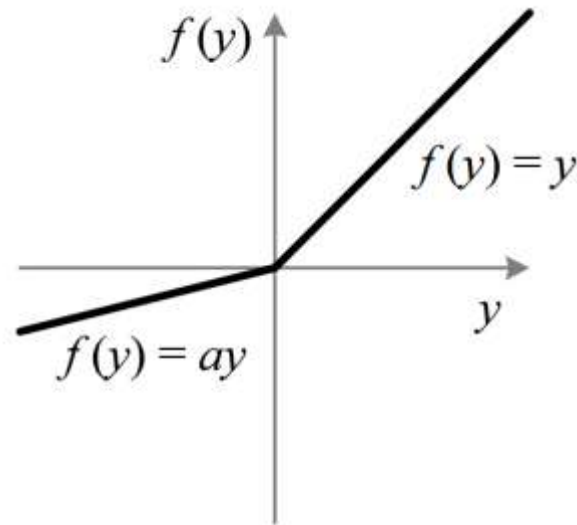


$$f(x) = \ln[1 + e^x]$$

Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

LeakyReLU



Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

softmax

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \text{ for } j = 1, \dots, K$$

Step 3) Define model architecture (Alternatives for activation)

```
model.add(Dense(128, activation='relu', use_bias=True))
```

Step 3) Define model architecture (Other attributes of Dense layer)

```
keras.layers.core.Dense(units, activation=None, use_bias=True,  
                        kernel_initializer='glorot_uniform',  
                        bias_initializer='zeros',  
                        kernel_regularizer=None,  
                        bias_regularizer=None,  
                        activity_regularizer=None,  
                        kernel_constraint=None,  
                        bias_constraint=None)
```

Step 3) Define model architecture

(Other attributes of Dense layer)

```
keras.layers.core.Dense(units, activation=None, use_bias=True,  
                        kernel_initializer='glorot_uniform',  
                        bias_initializer='zeros',  
                        kernel_regularizer=None,  
                        bias_regularizer=None,  
                        activity_regularizer=None,  
                        kernel_constraint=None,  
                        bias_constraint=None)
```

instances of
keras.regularizers.Regularizer
(l1, l2, ...)

Step 3) Define model architecture (Other attributes of Dense layer)

```
keras.layers.core.Dense(units, activation=None, use_bias=True,  
                        kernel_initializer='glorot_uniform',  
                        bias_initializer='zeros',  
                        kernel_regularizer=None,  
                        bias_regularizer=None,  
                        activity_regularizer=None,  
                        kernel_constraint=None,  
                        bias_constraint=None)
```

Functions from the **constraints** module allow setting constraints (eg. non-negativity) on network parameters during optimization

Example:

```
from keras.constraints import maxnorm  
model.add(Dense(64, kernel_constraint=max_norm(2.)))
```

Available constraints

max_norm(max_value=2, axis=0): maximum-norm constraint

non_neg(): non-negativity constraint

unit_norm(): unit-norm constraint, enforces the matrix to have unit norm along the last axis

Step 3) Define model architecture (Other attributes of Dense layer)

```
keras.layers.core.Dense(units, activation=None, use_bias=True,  
                        kernel_initializer='glorot_uniform',  
                        bias_initializer='zeros',  
                        kernel_regularizer=None,  
                        bias_regularizer=None,  
                        activity_regularizer=None,  
                        kernel_constraint=None,  
                        bias_constraint=None)
```

Step 3) Define model architecture (Dropout Layers)

```
keras.layers.core.Dropout(rate,  
                           noise_shape=None,  
                           seed=None)
```

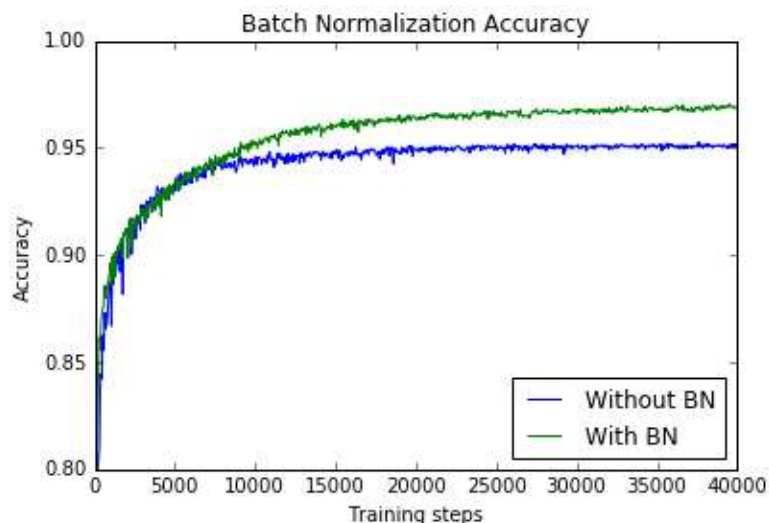
Example:

```
model.add(Dense(128, activation='relu', use_bias=True))  
model.add(Dropout(0.2))
```

Step 3) Define model architecture (Batch Normalization Layers)

Example:

```
model = Sequential()  
model.add(Dense(64, input_dim=14))  
model.add(BatchNormalization())  
model.add(Activation('tanh'))  
model.add(Dropout(0.5))
```



Input: Values of x over a mini-batch: $B = \{x_{1...m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Step 4) Compile model (Loss functions)

```
model.compile(loss='mean_squared_error',  
              optimizer='sgd',  
              metrics=['accuracy'])
```

Available loss functions:

- mean_squared_error
- mean_absolute_error
- mean_absolute_percentage_error
- mean_squared_logarithmic_error
- squared_hinge
- hinge
- categorical_hinge
- logcosh
- categorical_crossentropy
- sparse_categorical_crossentropy
- binary_crossentropy
- kullback_leibler_divergence
- poisson
- cosine_proximity

Step 4) Compile model (Loss functions)

```
model.compile(loss='mean_squared_error',  
              optimizer='sgd',  
              metrics=['accuracy'])
```

Custom loss function

```
import theano.tensor as T  
  
def myLoss(y_true, y_pred):  
    cce = T.mean(T.sqr(y_true-y_pred))  
    return cce
```

```
model.compile(optimizer='adadelta', loss=myLoss)
```

Available loss functions:

- mean_squared_error
- mean_absolute_error
- mean_absolute_percentage_error
- mean_squared_logarithmic_error
- squared_hinge
- hinge
- categorical_hinge
- logcosh
- categorical_crossentropy
- sparse_categorical_crossentropy
- binary_crossentropy
- kullback_leibler_divergence
- poisson
- cosine_proximity

Step 4) Compile model (Optimizers)

```
model.compile(loss='mean_squared_error',  
              optimizer='sgd',  
              metrics=['accuracy'])
```

Available loss functions:

- SGD
- RMSprop
- Adagrad
- Adadelta
- Adam
- Adamax
- Nadam
- TFOptimizer

Adagrad

```
adagrad = keras.optimizers.Adagrad(lr=0.01, epsilon=1e-08, decay=0.0)
```

```
model.compile(optimizer=adagrad, loss=myLoss)
```

